EN 649 Project: Concepts Guide and Kick-Start Plan

ML surrogate for DEM granular compaction (LIGGGHTS, GNS, PyTorch Geometric). Tailored to a laptop with an RTX 4060 (8 GB) and a short deadline.

1. The big picture

Simulating sand, soil, or powder grain by grain is accurate but slow. This is done with the **Discrete Element Method (DEM)**. Your project trains a **neural network surrogate** on DEM simulation data so it can predict the result (how densely the material compacts) much faster than running DEM. You compare it against an existing published ML simulator (**GNS**) and show the speed and accuracy trade-off.

2. Core concepts

Granular materials. Collections of discrete particles (sand, powder, gravel). They behave like neither a solid nor a liquid and are hard to model with continuum equations, so each grain is tracked instead.

DEM. Every particle is a sphere with a position and velocity. At each tiny timestep:

1. Find which particles touch (**contact detection**).
2. Compute forces from each contact: a **normal force** (along the line between centers) and a **tangential force** (friction, along the surface).
3. Sum forces, apply Newton's second law (F = ma), and update positions and velocities.
4. Repeat for millions of timesteps.

Steps 1 and 4 are what make DEM expensive. Your claim that the ML model "bypasses iterative contact detection" is the speed argument.

LIGGGHTS. An open-source DEM package built on LAMMPS. You write an input script defining the particles, the container, the contact model, and the compression, and it writes particle data over time.

Cohesion and the JKR/DMT models. Plain sand has no stickiness. Wet sand and fine powders do, from capillary or van der Waals forces. JKR (Johnson-Kendall-Roberts) and DMT (Derjaguin-Muller-Toporov) are contact models that add an attractive force between touching particles. JKR suits soft, strongly adhesive contacts; DMT suits stiffer, weakly

adhesive ones. Your **cohesion parameter** is essentially the surface energy: higher means stickier, which means looser packing and different compaction behavior. It is the key variable you sweep.

Compaction, density, and void ratio. You compress the particle bed with a piston. Two numbers describe the result:

- **Void ratio** e = V_voids / V_solids
- **Bulk density** = mass / total volume

More compression gives a lower void ratio and higher density. Your model's job is to predict these final values.

Force vs. displacement curve. Piston force plotted against piston travel. It is the standard way to characterize compaction, and your Week 7 plots compare DEM and ML on it.

Graph Neural Networks (GNNs). Particles become **nodes**. Particles close enough to interact get an **edge**. Each node carries features (position, velocity, type) and each edge can carry features (distance, relative position). The GNN learns by **message passing**:

1. Each edge computes a message from its two endpoint nodes.
2. Each node aggregates the messages from its neighbors (sum or mean).
3. Each node updates its own state.
4. Repeat for several layers so information spreads further.

This matches physics well: forces are local, and the same rules apply to every particle (shared weights, permutation invariance).

GNS (Graph Network Simulator). DeepMind's model (Sanchez-Gonzalez et al., 2020). It predicts the next-step acceleration of each particle from the last few timesteps, then **rolls out** step by step to simulate a full trajectory. The DesignSafe datasets (Sand, SandRamps, WaterDropSample) come from the geoelements adaptation of GNS for granular flows, so open-source code exists (the `geoelements/gns` GitHub repo).

PyTorch Geometric (PyG). A GNN library on top of PyTorch. It provides graph data structures (`Data`), neighbor finding (`radius_graph`), and layers (`MessagePassing` , `GCNConv` , `GATConv` , `NNConv`). Your custom GNN is built from these.

Custom model vs. GNS (important distinction).

- GNS is **autoregressive**: many small steps, and errors accumulate.

- Your custom model is **direct**: initial particle state in, final density and void ratio out. This is graph-level regression. A pooling (readout) layer collapses all node embeddings into one vector, and an MLP maps it to the targets.

Because they predict different things, benchmark carefully. Run the GNS rollout to the final timestep, compute density and void ratio from its final positions, and compare both models against the DEM ground truth.

.npz format. NumPy's compressed archive. GNS expects each trajectory as a tuple of an array of positions with shape `[timesteps, n_particles, 3]` and an array of particle types with shape `[n_particles]`. It also needs a `metadata.json` with bounds, timestep, connectivity radius, and the mean and standard deviation of velocity and acceleration for normalization.

Evaluation metrics. MAE, RMSE, and relative error on final density and void ratio; R^2 on a held-out test set; wall-clock time per prediction vs. per DEM run.

3. Reality check

Eight weeks of work will not fit in the time you have at full scale. Scope down and list the rest as future work in the report.

Proposal	Scaled-down version
50 to 200 LIGGGHTS runs	About 30 to 50 small runs (300 to 1000 particles, short compression)
Full GNS training on your data	Train a few thousand steps, or only validate the baseline on the DesignSafe data
Elaborate architecture iteration	One baseline GNN plus one improved variant
Multiple materials	2 or 3 cohesion levels, called "materials"

4. Your setup: laptop with RTX 4060 (8 GB)

The 4060 is plenty for this scope. A GNN on a few hundred to a few thousand particles trains in minutes, and the GNS baseline at small scale fits in 8 GB if you keep batch size at 1 and particle counts modest. The bottleneck is DEM data generation, which runs on the CPU, not the GPU.

If you are on Windows, use WSL2 (Ubuntu). LIGGGHTS builds on Linux. Open PowerShell as administrator and run `wsl --install`, restart, then work inside Ubuntu. PyTorch in WSL2 can still use your GPU through the normal NVIDIA Windows driver.

5. Kick-start steps (do these in order)

Step 1: Environment (30 to 45 min)

1. Install WSL2 + Ubuntu (Windows) and confirm the GPU with `nvidia-smi` inside Ubuntu.
2. Install build tools: `sudo apt update && sudo apt install -y build-essential git cmake mpi-default-bin mpi-default-dev libvtk7-dev python3-pip python3-venv`
3. Create a Python environment and install the libraries: `python3 -m venv ~/env && source ~/env/bin/activate`, then `pip install torch numpy scipy matplotlib pandas`, then `pip install torch_geometric`. For `radius_graph` you also need `torch-cluster`; if it fails to install, use `scipy.spatial.cKDTree` to build edges instead.

Step 2: Get a DEM engine running (timebox: 45 min)

- **Try LIGGGHTS first:** `git clone https://github.com/CFDEMproject/LIGGGHTS-PUBLIC.git`, then `cd LIGGGHTS-PUBLIC/src && make auto -j8`. Run one of the bundled examples to confirm it works.
- **Important caveat:** LIGGGHTS-PUBLIC's built-in cohesion models are SJKR-type (simplified JKR). True JKR and DMT contact models are in **LAMMPS** (`pair_style granular`). For your report, state which you used.
- **Fallback if the build fails or the timebox runs out:** install LAMMPS (`conda install -c conda-forge lammps`, or `sudo apt install lammps`) and use `pair_style granular` with the `jkr` or `dmt` normal model. Note the substitution in the report. This is a legitimate, defensible choice.

Step 3: One simulation that works (1 to 1.5 hr)

Write one input script that does the following:

1. Create a box (container) and insert a few hundred to a thousand spheres with random positions (a loose bed).

2. Define the contact model with a cohesion parameter you can change from the command line (a variable).
3. Let the bed settle under gravity.
4. Move a top wall (piston) downward by a set displacement.
5. Dump particle positions, velocities, and forces every N steps to a file, and log the wall force.

Run it once and check it visually (plot the dump file or use OVITO). Do not scale up until this single run looks physically sensible.

Step 4: Sweep driver (45 min)

Write a Python script that loops over cohesion values (for example 3 levels), displacement or stress values (for example 3 to 4 levels), and random seeds (for example 3 to 4), launches each run, and stores results in separate folders. Run it in the background (`nohup python sweep.py &`) and keep working while it runs. Aim for 30 to 50 total runs. Use as many CPU cores as you have by running several single-core simulations in parallel.

Step 5: Extract data to .npz (1 hr)

For each run, parse the dump file and save `positions` (timesteps x particles x 3) and `particle_types`. Also compute the labels: **final density** and **final void ratio**, plus the force vs. displacement curve. Write a `metadata.json` (bounds, timestep, connectivity radius, velocity and acceleration statistics).

Step 6: Models (about 3 to 4 hr)

1. **Custom PyG model:** build the graph with `radius_graph` on initial positions, encode node and edge features with small MLPs, apply 3 to 5 message-passing layers, apply global mean pooling, then an MLP to output (density, void ratio). Train with MSE and an 80/10/10 train/validation/test split. Add the cohesion value and applied displacement as global inputs.
2. **Improved variant:** add edge features (relative displacement and distance) and a residual connection or attention, then compare against the baseline.
3. **GNS baseline:** clone `geoelements/gns`, validate on the public DesignSafe data, and train briefly on your `.npz` data in the background on the GPU.

Step 7: Benchmark and report (2 to 3 hr)

Produce an error table (MAE, RMSE, R^2) for the custom model, the improved variant, and GNS; a timing table (DEM run vs. surrogate inference); predicted vs. true scatter plots; and force vs. displacement plots for DEM vs. ML. Then write up method, results, limitations, and future work.

6. Suggested time allocation for the next ~24 hours

Block	Task
Hours 0 to 1	Environment, WSL2, install DEM engine
Hours 1 to 3	Working single simulation
Hours 3 to 4	Sweep driver, launch batch runs
Hours 4 to 6	While runs go: download DesignSafe data, clone GNS, write extraction script
Hours 6 to 10	Build and train the PyG models
Hours 10 to 13	Benchmark, plots, GNS validation
Hours 13 to 16	Write the report

7. Risks and how to handle them

- **DEM too slow:** reduce particle count, increase particle size, shorten the compression, or use fewer runs.
- **Install problems:** use the LAMMPS fallback and say so in the report.
- **GNS training does not converge in time:** report the validation of the pretrained or baseline setup on DesignSafe data and show partial training curves on your data. Be explicit that it is partial.
- **Small dataset (about 30 to 50 runs):** keep the model small, use weight decay, and report test error honestly with a held-out split.

8. What to send back for help

When you reach each step, paste the error message or output and ask for the next piece. The most useful first piece of code is the simulation input script and sweep driver, since everything else depends on that data.